

WEEK 3 ASSIGNMENT

Title:

SQL Advanced Analytics using Subqueries, CTEs and Window Functions

Submitted By:

Siga Nithin

Domain:

Data Engineering

1. OBJECTIVE

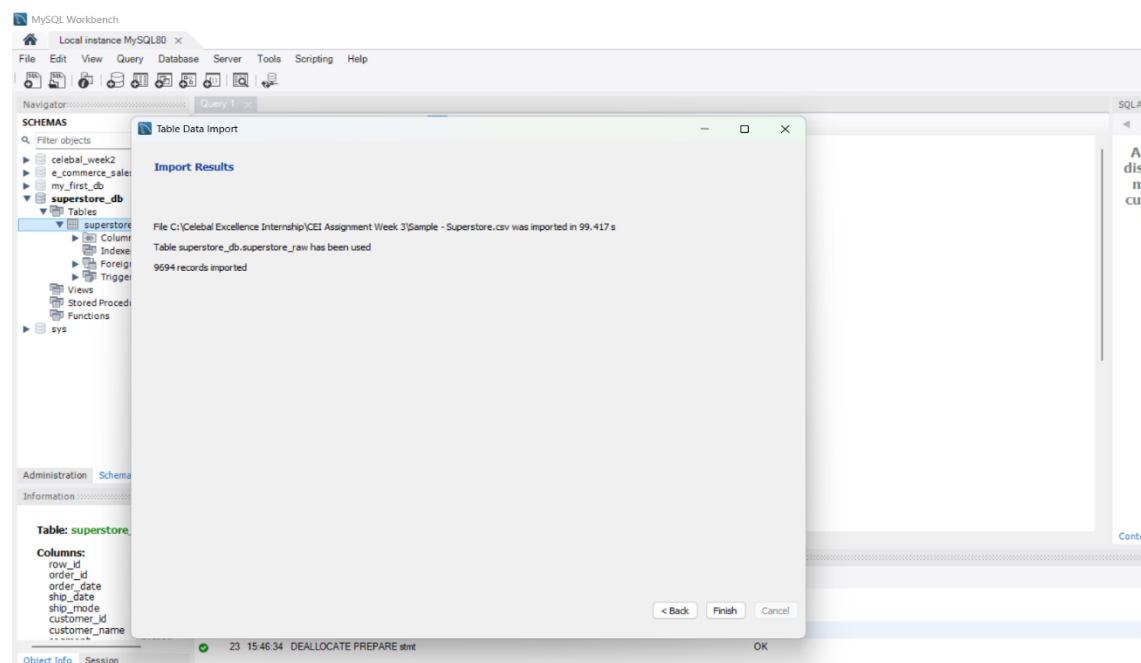
The objective of this assignment is to analyze the Superstore dataset using advanced SQL concepts such as Subqueries, Common Table Expressions (CTEs), Window Functions, and Joins. The dataset was normalized into separate tables and used to answer business-oriented sales questions.

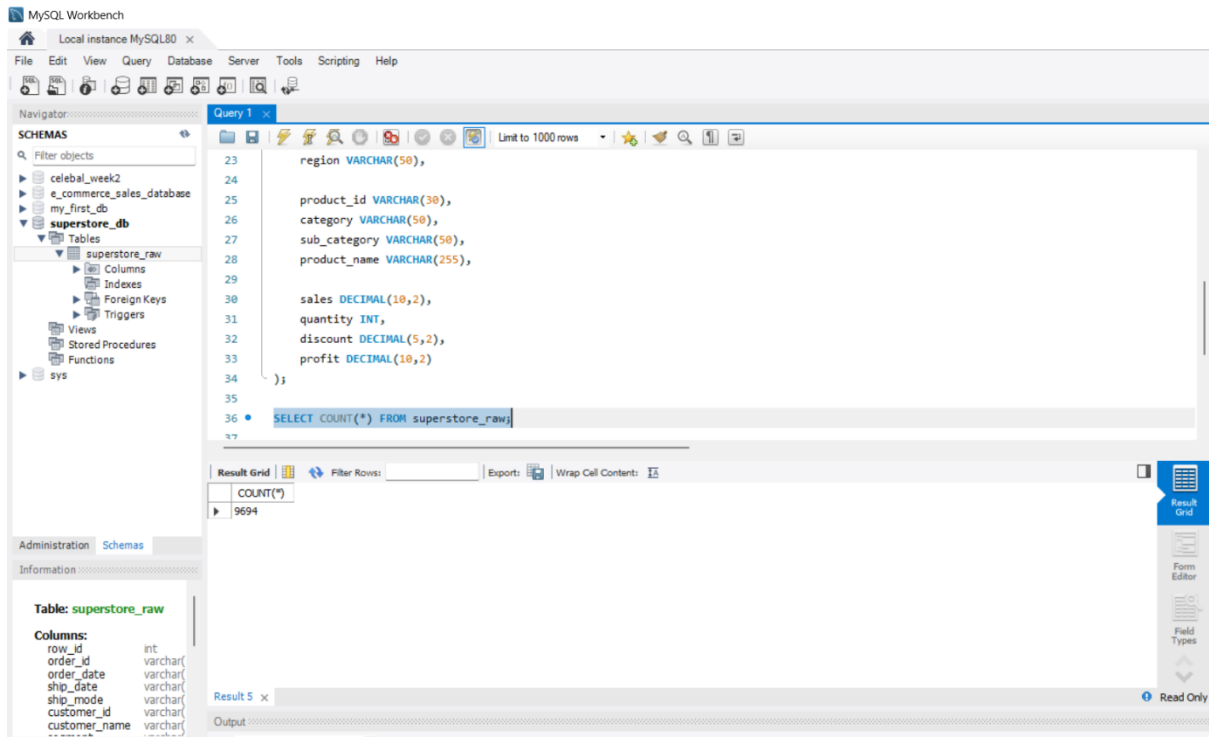
2. DATASET OVERVIEW

The Superstore dataset contains customer, product, and sales transaction information. The dataset was first imported into a raw table and later normalized into separate tables to reduce redundancy and improve query performance.

Tables Created:

- superstore_raw
- customers
- products
- orders





3.DATA NORMALIZATION

The raw dataset contained customer, product, and order information in a single table. To organize the data better and reduce repetition, the dataset was divided into separate tables.

The data was split into customers, products, and orders tables so that each table stores a specific type of information.

Customers Table:

The customers table was created to store unique customer records. Customer information appeared multiple times in the raw dataset because the same customer could place multiple orders.

Using DISTINCT helped extract only unique customer records.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'superstore_db' selected. The main editor shows a SQL script with the following queries:

```

30 sales DECIMAL(10,2),
31 quantity INT,
32 discount DECIMAL(5,2),
33 profit DECIMAL(10,2)
34 );
35
36 SELECT COUNT(*) FROM superstore_raw;
37
38
39 SELECT * FROM superstore_raw;
40
41
42 -- Creating customers table
43 CREATE TABLE customers (
44   customer_id VARCHAR(20) PRIMARY KEY,
45   customer_name VARCHAR(100),
46   segment VARCHAR(50)
47 );
48
49
50
51
52

```

The 'Output' pane at the bottom shows the results of the executed queries:

#	Time	Action	Message
26	15:50:48	SELECT * FROM superstore_raw LIMIT 0, 1000	1000 row(s) returned
27	15:51:08	SELECT COUNT(*) FROM superstore_raw LIMIT 0, 1000	1 row(s) returned
28	15:51:21	SELECT * FROM superstore_raw LIMIT 0, 1000	1000 row(s) returned
29	15:53:01	CREATE TABLE customers (customer_id VARCHAR(20) PRIMARY KEY, customer_name VARCHAR(100), segment VARCHAR(50))	0 row(s) affected

The screenshot shows the MySQL Workbench interface with the 'customers' table created. The main editor shows the following queries:

```

51 SELECT DISTINCT
52   customer_id,
53   customer_name,
54   segment
55 FROM superstore_raw;
56
57 SELECT COUNT(*) FROM customers;
58
59 SELECT * FROM customers LIMIT 10;
60
61
62
63

```

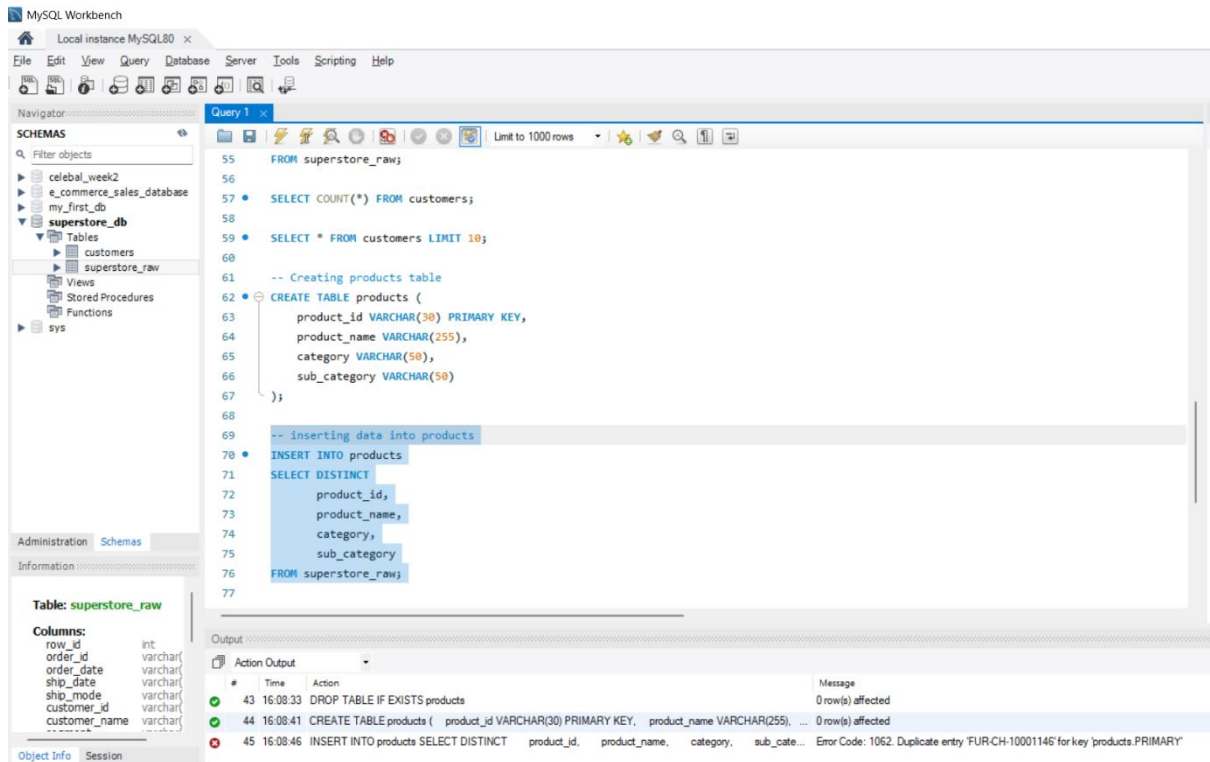
The 'Result Grid' pane at the bottom displays the results of the last query (SELECT * FROM customers LIMIT 10):

customer_id	customer_name	segment
AA-10315	Alex Avila	Consumer
AA-10375	Allen Arnold	Consumer
AA-10480	Andrew Allen	Consumer
AA-10645	Anna Andreadi	Consumer
AB-10015	Aaron Bergman	Consumer
AB-10060	Adam Bellavance	Home Office
AB-10105	Adrian Barton	Consumer
AB-10150	Aimee Bixby	Consumer
AB-10165	Alan Barnes	Consumer
AB-10255	Alejandro Ballentine	Home Office

Products Table

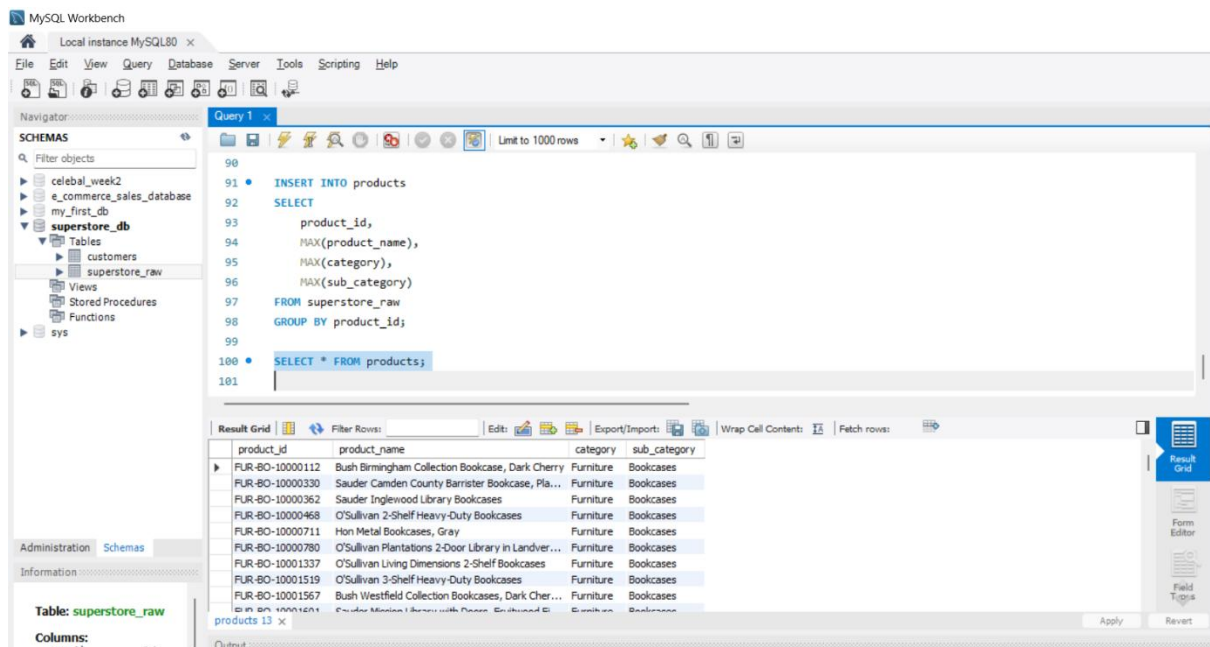
The products table was created to store product-related information separately.

During data preparation, it was observed that some Product IDs appeared with multiple product descriptions in the raw dataset.



To avoid duplicate Product IDs in the final table, the records were grouped using Product ID before insertion.

This ensured that each product appeared only once in the products table.



Orders Table

The orders table stores transaction-level information such as sales, quantity, discount, and profit.

Unlike customers and products, every order record is important because each row represents a sales transaction. Therefore, all order records were inserted into the table.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'superstore_db' selected. The main editor shows a SQL query window with the following code:

```
92 SELECT
93     product_id,
94     MAX(product_name),
95     MAX(category),
96     MAX(sub_category)
97 FROM superstore_raw
98 GROUP BY product_id;
99
100 • SELECT * FROM products;
101
102 -- creating the orders table
103 • CREATE TABLE orders (
104     row_id INT,
105     order_id VARCHAR(30),
106     order_date VARCHAR(20),
107     customer_id VARCHAR(20),
108     product_id VARCHAR(30),
109     sales DECIMAL(10,2),
110     quantity INT,
111     discount DECIMAL(5,2),
112     profit DECIMAL(10,2)
113 );
114
```

The bottom panel shows the 'Output' tab with the following log entries:

#	Time	Action	Message
48	16:11:04	INSERT INTO products SELECT product_id, MAX(product_name), MAX(category), MAX(sub_category)	1812 row(s) affected Records: 1812 Duplicates: 0 Warnings: 0
49	16:11:23	SELECT * FROM products LIMIT 0.1000	1000 row(s) returned
50	16:13:40	CREATE TABLE orders (row_id INT, order_id VARCHAR(30), order_date VARCHAR(20), customer_id VARCHAR(20), product_id VARCHAR(30), sales DECIMAL(10,2), quantity INT, discount DECIMAL(5,2), profit DECIMAL(10,2))	0 row(s) affected

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'superstore_db' selected. The main editor shows a SQL query window with the following code:

```
118     order_id,
119     order_date,
120     customer_id,
121     product_id,
122     sales,
123     quantity,
124     discount,
125     profit
126 FROM superstore_raw;
127
128 • SELECT COUNT(*) AS total_orders
129 FROM orders;
130
131 • SELECT * FROM orders
132 LIMIT 10;
```

The bottom panel shows the 'Result Grid' tab with the following data:

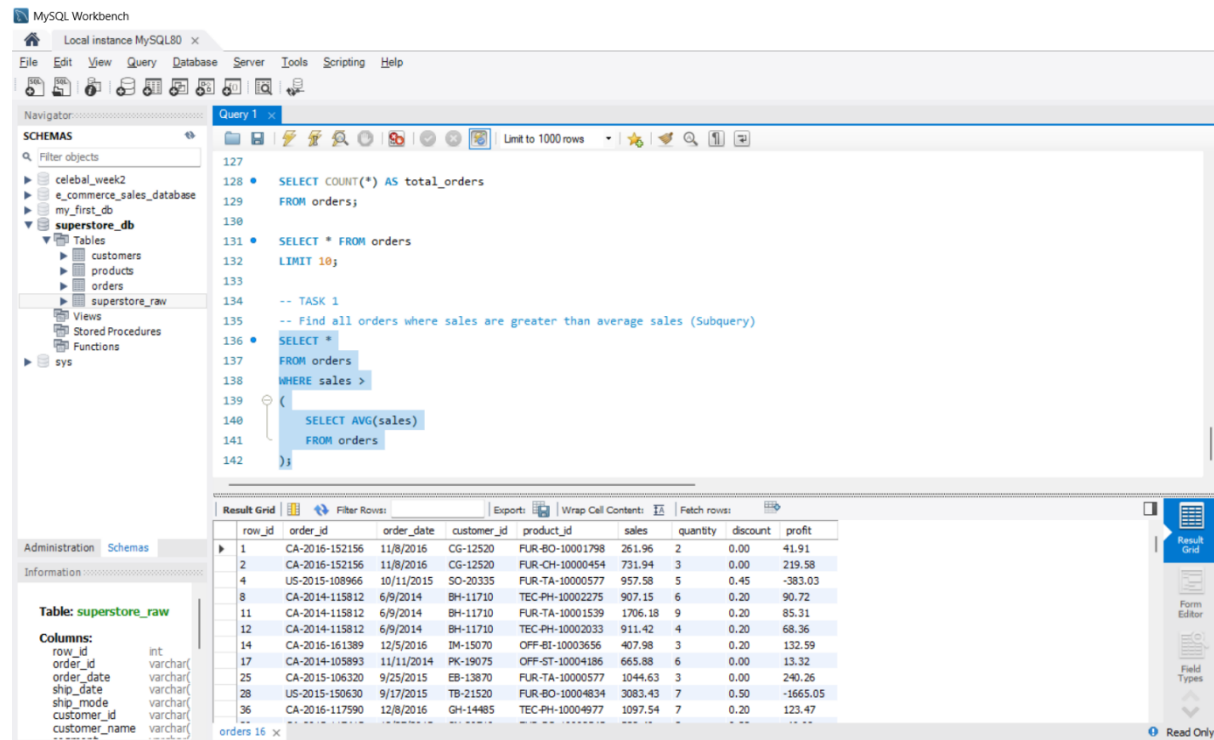
row_id	order_id	order_date	customer_id	product_id	sales	quantity	discount	profit
1	CA-2016-152156	11/8/2016	CG-12520	FUR-BO-10001798	261.96	2	0.00	41.91
2	CA-2016-152156	11/8/2016	CG-12520	FUR-CH-10000454	731.94	3	0.00	219.58
3	CA-2016-138688	6/12/2016	DV-13045	OFF-LA-10000240	14.62	2	0.00	6.87
4	US-2015-108966	10/11/2015	SO-20335	FUR-TA-10000577	957.58	5	0.45	-383.03
5	US-2015-108966	10/11/2015	SO-20335	OFF-ST-10000760	22.37	2	0.20	2.52

4. SQL ANALYSIS

Query 1: Orders with Sales Greater than Average Sales

I first calculated the average sales value across all orders and then filtered only those orders whose sales were higher than the average.

This helped identify transactions that performed better than a typical order in the dataset.



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'superstore_db' selected. The main editor shows a SQL query for 'Query 1' with line numbers 127 to 142. The query calculates the total number of orders and then filters for orders with sales greater than the average sales value.

```
127
128 • SELECT COUNT(*) AS total_orders
129 FROM orders;
130
131 • SELECT * FROM orders
132 LIMIT 10;
133
134 -- TASK 1
135 -- Find all orders where sales are greater than average sales (Subquery)
136 • SELECT *
137 FROM orders
138 WHERE sales >
139 (
140     SELECT AVG(sales)
141     FROM orders
142 )
```

The 'Result Grid' at the bottom shows a table with 16 rows and 9 columns: row_id, order_id, order_date, customer_id, product_id, sales, quantity, discount, and profit. The data is sorted by sales in descending order.

row_id	order_id	order_date	customer_id	product_id	sales	quantity	discount	profit
1	CA-2016-152156	11/8/2016	CG-12520	FUR-BO-10001798	261.96	2	0.00	41.91
2	CA-2016-152156	11/8/2016	CG-12520	FUR-CH-10000454	731.94	3	0.00	219.58
4	US-2015-108966	10/11/2015	SO-20335	FUR-TA-10000577	957.58	5	0.45	-383.03
8	CA-2014-115812	6/9/2014	BH-11710	TEC-PH-10002275	907.15	6	0.20	90.72
11	CA-2014-115812	6/9/2014	BH-11710	FUR-TA-10001539	1706.18	9	0.20	85.31
12	CA-2014-115812	6/9/2014	BH-11710	TEC-PH-10002033	911.42	4	0.20	68.36
14	CA-2016-161389	12/5/2016	IM-15070	OFF-BI-10003656	407.98	3	0.20	132.59
17	CA-2014-105893	11/11/2014	PK-19075	OFF-ST-10004186	665.88	6	0.00	13.32
25	CA-2015-106320	9/25/2015	EB-13870	FUR-TA-10000577	1044.63	3	0.00	240.26
28	US-2015-150630	9/17/2015	TB-21520	FUR-BO-10004834	3083.43	7	0.50	-1665.05
36	CA-2016-117590	12/8/2016	GH-14485	TEC-PH-10004977	1097.54	7	0.20	123.47

Observation: The result contains orders with significantly higher sales values compared to the overall average sales.

Query 2: Highest Order Value for Each Customer

To understand the largest purchase made by each customer, I calculated the maximum sales value for every customer.

This provides a quick view of each customer's highest-value transaction.

Query 1

```

138 WHERE sales >
139 (
140     SELECT AVG(sales)
141     FROM orders
142 );
143
144 -- Find the highest sales order for each customer (Subquery)
145 SELECT
146     customer_id,
147     MAX(sales) AS highest_order_value
148 FROM orders
149 GROUP BY customer_id;

```

Result Grid

customer_id	highest_order_value
CG-12520	731.94
DV-13045	721.88
SO-20335	957.58
BH-11710	1706.18
AA-10480	479.97

Table: superstore_raw

Columns:

- row_id int
- order_id varchar
- order_date varchar
- ship_date varchar
- ship_mode varchar
- customer_id varchar
- customer_name varchar

Action Output

#	Time	Action	Message
52	16:14:21	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned
53	16:14:53	SELECT * FROM orders LIMIT 10	10 row(s) returned
54	16:19:13	SELECT * FROM orders WHERE sales > (SELECT AVG(sales) FROM orders) LIMIT 0, 1000	1000 row(s) returned
55	16:19:59	SELECT * FROM orders o WHERE sales = (SELECT MAX(sales) FROM orders WHERE customer_id ...	Error Code: 2013. Lost connection to MySQL server during query
56	16:20:47	SELECT * FROM orders o WHERE sales = (SELECT MAX(sales) FROM orders WHERE customer_id ...	Error Code: 2013. Lost connection to MySQL server during query
57	16:21:46	SELECT * FROM orders o WHERE sales = (SELECT MAX(sales) FROM orders WHERE customer_id ...	Error Code: 2013. Lost connection to MySQL server during query
58	16:23:54	SELECT customer_id, MAX(sales) AS highest_order_value FROM orders GROUP BY customer_id LIMIT ...	793 row(s) returned

Observation: Every customer has a different maximum order value, showing variation in purchasing behaviour.

Query 3: Total Sales for Each Customer (CTE)

- A Common Table Expression (CTE) was used to calculate the total sales generated by each customer.
- Using a CTE made the query easier to read and reuse in later analysis.

Query 1

```

150
151 -- Calculate Total Sales for Each Customer (CTE)
152 WITH customer_sales AS
153 (
154     SELECT
155         customer_id,
156         SUM(sales) AS total_sales
157     FROM orders
158     GROUP BY customer_id
159 )
160 SELECT *
161 FROM customer_sales;

```

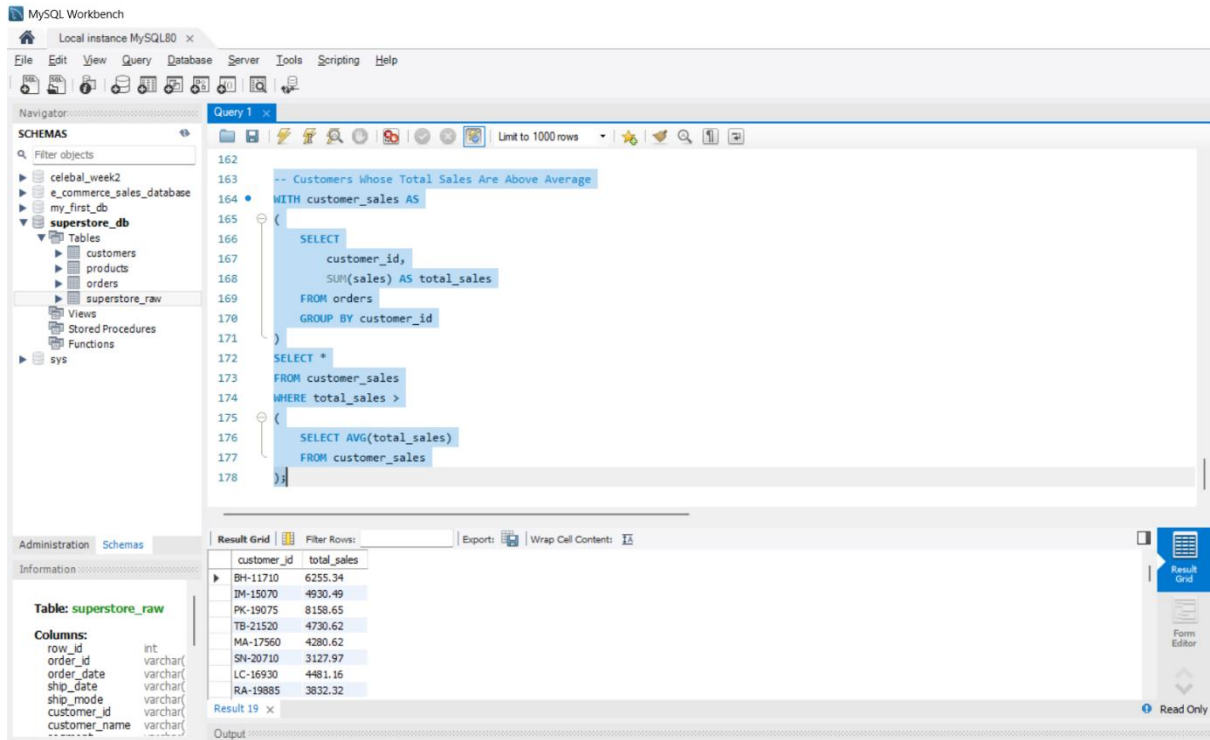
Result Grid

customer_id	total_sales
CG-12520	1148.78
DV-13045	1119.48
SO-20335	2602.58
BH-11710	6255.34
AA-10480	1782.87

Observation: The total sales generated by customers vary significantly across the dataset.

Query 4: Customers with Above Average Sales

- After calculating total sales for each customer, I compared those values with the average customer sales.
- This helped identify customers who contributed more revenue than the average customer.



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'superstore_db' selected. The main editor window shows a SQL query for 'Query 1'. The query calculates the total sales for each customer and compares them to the average total sales. The 'Result Grid' at the bottom shows the results of the query, listing customer IDs and their total sales.

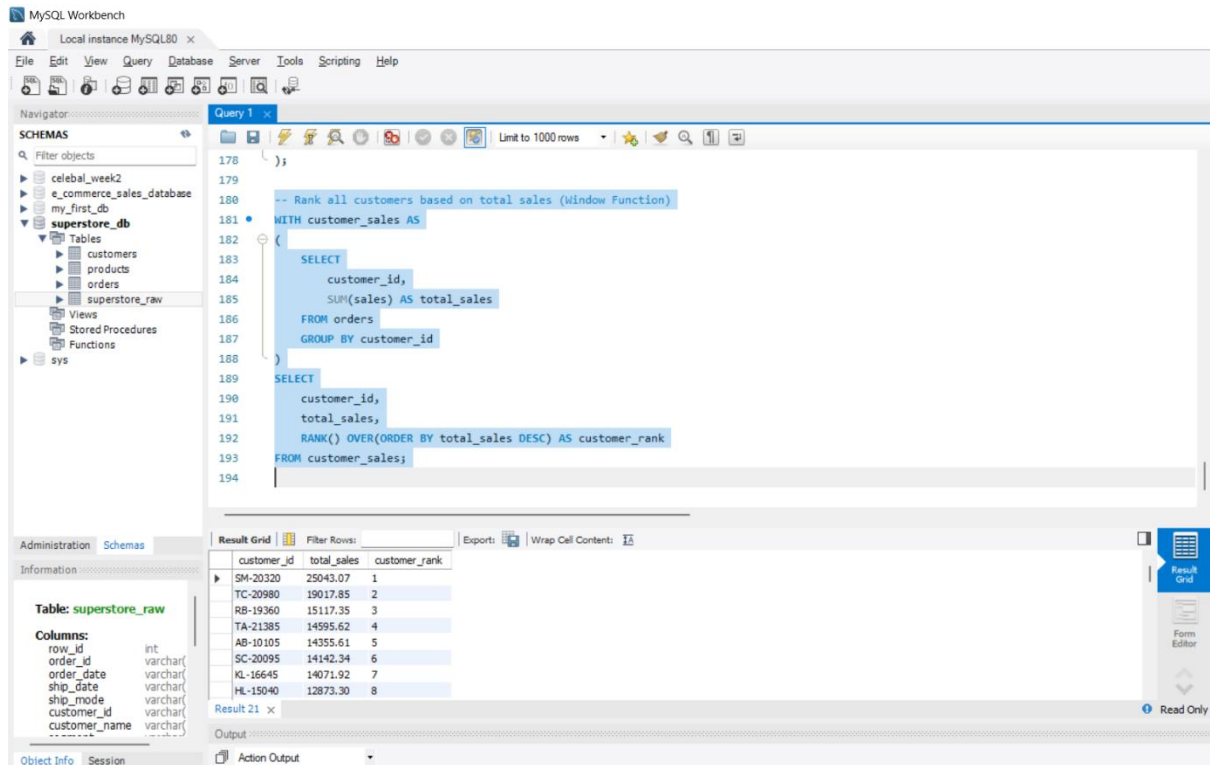
```
162
163 -- Customers Whose Total Sales Are Above Average
164 WITH customer_sales AS
165 (
166     SELECT
167         customer_id,
168         SUM(sales) AS total_sales
169     FROM orders
170     GROUP BY customer_id
171 )
172 SELECT *
173 FROM customer_sales
174 WHERE total_sales >
175 (
176     SELECT AVG(total_sales)
177     FROM customer_sales
178 )
```

customer_id	total_sales
BH-11710	6255.34
IM-15070	4930.49
PK-19075	8158.65
TB-21520	4730.62
MA-17560	4280.62
SN-20710	3127.97
LC-16930	4481.16
RA-19885	3832.32

Observation: Only a smaller group of customers generated sales above the average customer sales value.

Query 5: Customer Ranking using Window Function

- After calculating customer sales, I used the RANK() window function to rank customers based on total sales.
- Customers with higher sales received better ranks.



The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' tree with 'superstore_db' selected. The main editor window contains a SQL query (Query 1) that calculates total sales for each customer and ranks them. The query is as follows:

```
178 );
179
180 -- Rank all customers based on total sales (Window Function)
181 WITH customer_sales AS
182 (
183     SELECT
184         customer_id,
185         SUM(sales) AS total_sales
186     FROM orders
187     GROUP BY customer_id
188 )
189 SELECT
190     customer_id,
191     total_sales,
192     RANK() OVER(ORDER BY total_sales DESC) AS customer_rank
193 FROM customer_sales;
194
```

Below the query editor, the 'Result Grid' shows the output of the query. The columns are 'customer_id', 'total_sales', and 'customer_rank'. The results are as follows:

customer_id	total_sales	customer_rank
SM-20320	25043.07	1
TC-20980	19017.85	2
RB-19360	15117.35	3
TA-21385	14595.62	4
AB-10105	14355.61	5
SC-20095	14142.34	6
KL-16645	14071.92	7
HL-15040	12873.30	8

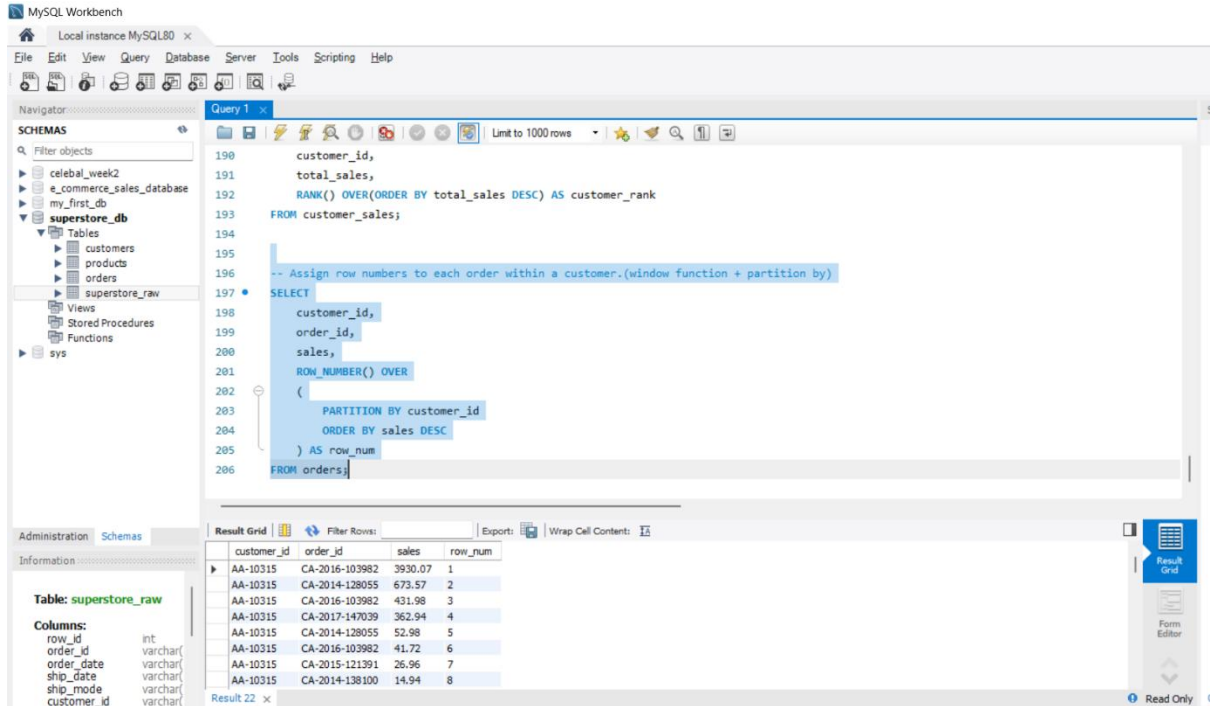
The bottom of the interface shows the 'Table: superstore_raw' with its columns and data types.

Observation:

The ranking shows a clear difference in sales contribution among customers. A few customers generated significantly higher sales compared to the rest.

Query 6: Row Number Assignment using PARTITION BY

- ROW_NUMBER() was used along with PARTITION BY to assign order numbers separately for each customer.
- This helps in analyzing the order sequence within a customer's transactions.



The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'superstore_db' database and its tables: 'customers', 'products', 'orders', and 'superstore_raw'. The main editor window shows a SQL query (Query 1) that assigns row numbers to orders within each customer's transactions. The query is as follows:

```
190 customer_id,  
191 total_sales,  
192 RANK() OVER(ORDER BY total_sales DESC) AS customer_rank  
193 FROM customer_sales;  
194  
195  
196 -- Assign row numbers to each order within a customer.(window function + partition by)  
197 SELECT  
198 customer_id,  
199 order_id,  
200 sales,  
201 ROW_NUMBER() OVER  
202 (  
203 PARTITION BY customer_id  
204 ORDER BY sales DESC  
205 ) AS row_num  
206 FROM orders;
```

The bottom of the screenshot shows the 'Result Grid' with the following data:

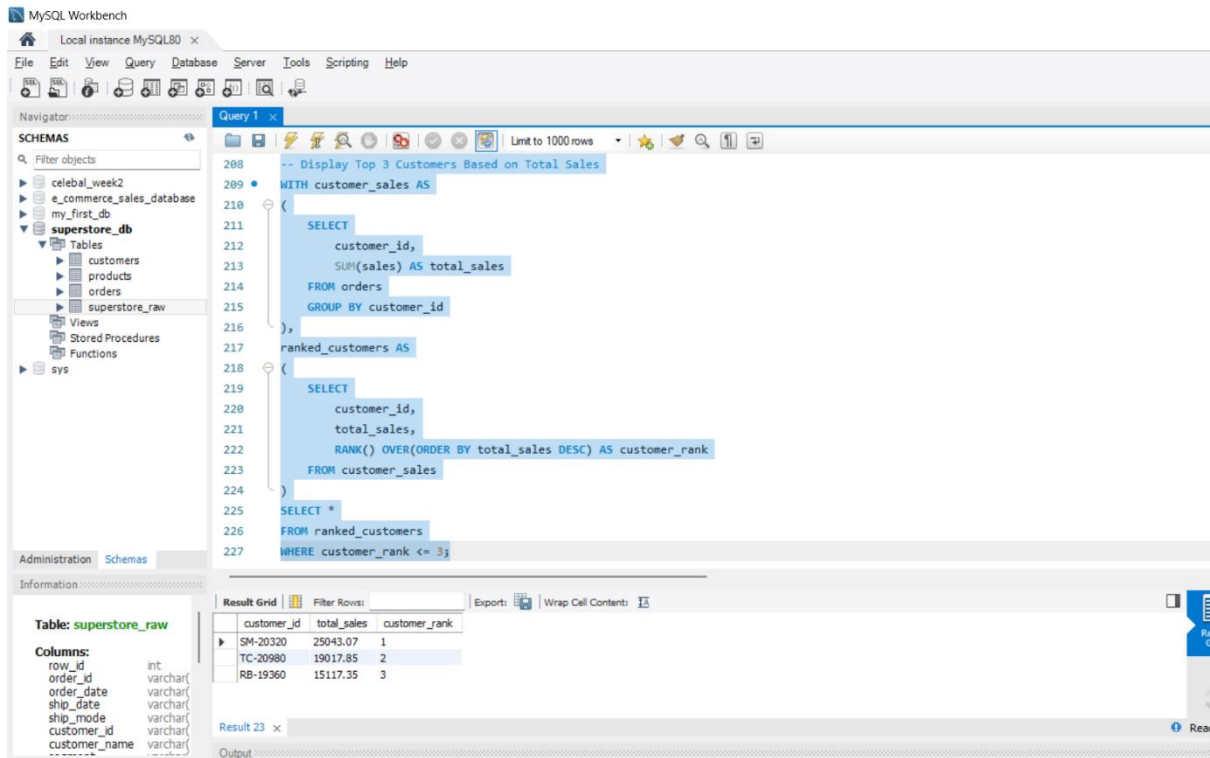
customer_id	order_id	sales	row_num
AA-10315	CA-2016-103982	3930.07	1
AA-10315	CA-2014-128055	673.57	2
AA-10315	CA-2016-103982	431.98	3
AA-10315	CA-2017-147039	362.94	4
AA-10315	CA-2014-128055	52.98	5
AA-10315	CA-2016-103982	41.72	6
AA-10315	CA-2015-121391	26.96	7
AA-10315	CA-2014-138100	14.94	8

Observation:

The numbering restarts for every customer, which confirms that PARTITION BY is working correctly.

Query 7: Top 3 Customers

- Using customer rankings, the top three customers were identified based on total sales.
- This provides a quick view of the highest revenue-generating customers.



The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the database structure, including tables like 'customers', 'products', 'orders', and 'superstore_raw'. The main editor shows a SQL query titled 'Query 1' with the following code:

```
-- Display Top 3 Customers Based on Total Sales
WITH customer_sales AS
(
  SELECT
    customer_id,
    SUM(sales) AS total_sales
  FROM orders
  GROUP BY customer_id
),
ranked_customers AS
(
  SELECT
    customer_id,
    total_sales,
    RANK() OVER(ORDER BY total_sales DESC) AS customer_rank
  FROM customer_sales
)
SELECT *
FROM ranked_customers
WHERE customer_rank <= 3;
```

Below the query editor, the 'Result Grid' shows the output of the query. It displays three columns: 'customer_id', 'total_sales', and 'customer_rank'. The results are as follows:

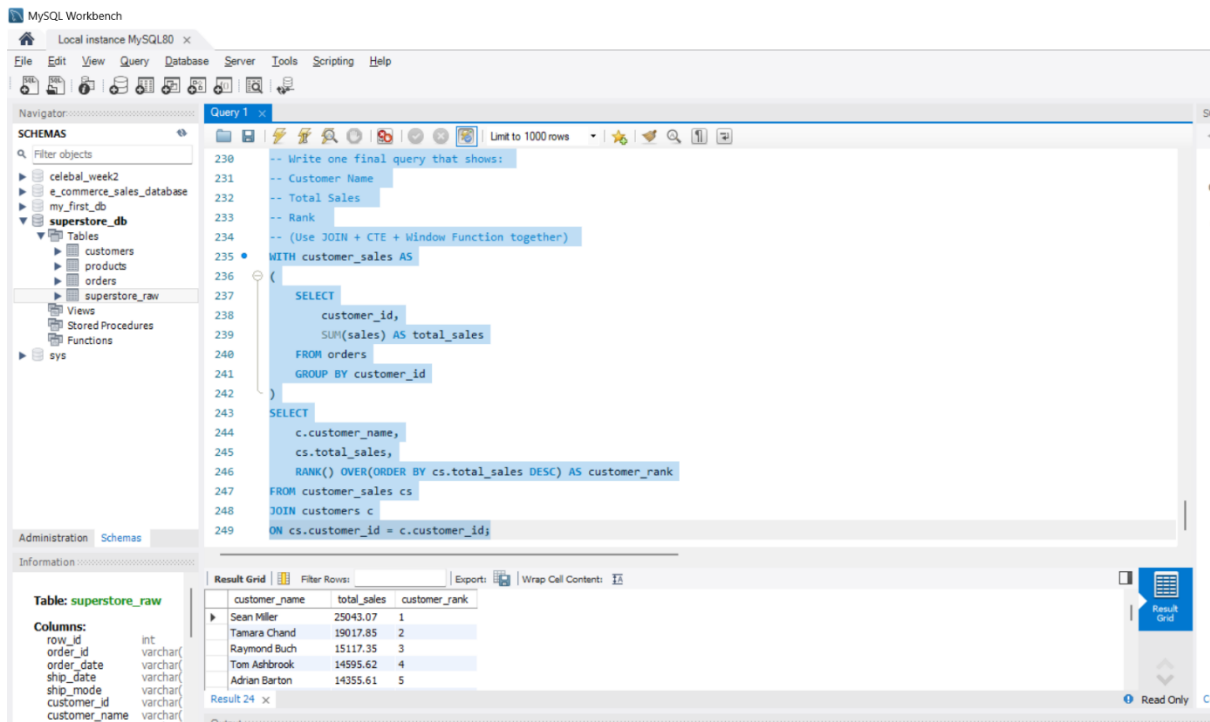
customer_id	total_sales	customer_rank
SM-20320	25043.07	1
TC-20980	19017.85	2
RB-19360	15117.35	3

Observation :

The top three customers generated noticeably higher sales than most customers in the dataset, indicating that revenue is concentrated among a small customer group.

5. FINAL COMBINED QUERY

- In the final query, customer information was combined with total sales and ranking details.
- This query brings together multiple SQL concepts including JOIN, CTE, and Window Functions to create a customer ranking report.



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'superstore_db' selected. The main editor shows a SQL query (Query 1) that uses a Common Table Expression (CTE) and a window function to rank customers by total sales. The query is as follows:

```
230 -- Write one final query that shows:
231 -- Customer Name
232 -- Total Sales
233 -- Rank
234 -- (Use JOIN + CTE + Window Function together)
235 WITH customer_sales AS
236 (
237     SELECT
238         customer_id,
239         SUM(sales) AS total_sales
240     FROM orders
241     GROUP BY customer_id
242 )
243 SELECT
244     c.customer_name,
245     cs.total_sales,
246     RANK() OVER(ORDER BY cs.total_sales DESC) AS customer_rank
247 FROM customer_sales cs
248 JOIN customers c
249 ON cs.customer_id = c.customer_id;
```

The bottom of the screenshot shows the 'Result Grid' with the following data:

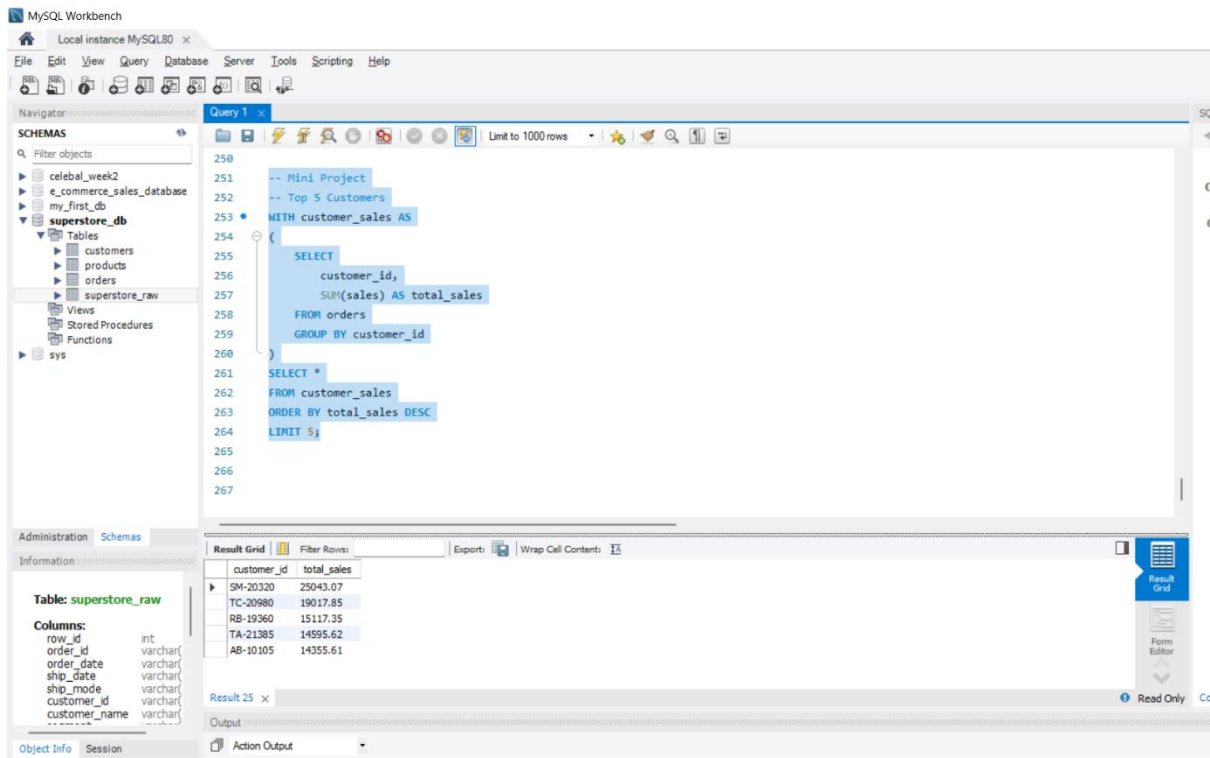
customer_name	total_sales	customer_rank
Sean Miller	25043.07	1
Tamara Chand	19017.85	2
Raymond Buch	15117.35	3
Tom Ashbrook	14595.62	4
Adrian Barton	14355.61	5

Observation:

This report combines customer details with sales performance and ranking information, making it easier to identify high-value customers and compare their contributions.

6. CUSTOMER SALES INSIGHTS

Top 5 Customers



The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays a tree view of databases and tables, with 'superstore_db' selected. The main editor window shows a SQL query for 'Query 1'. The query uses a CTE named 'customer_sales' to calculate the total sales for each customer from the 'orders' table, then ranks them in descending order of total sales and limits the results to the top 5.

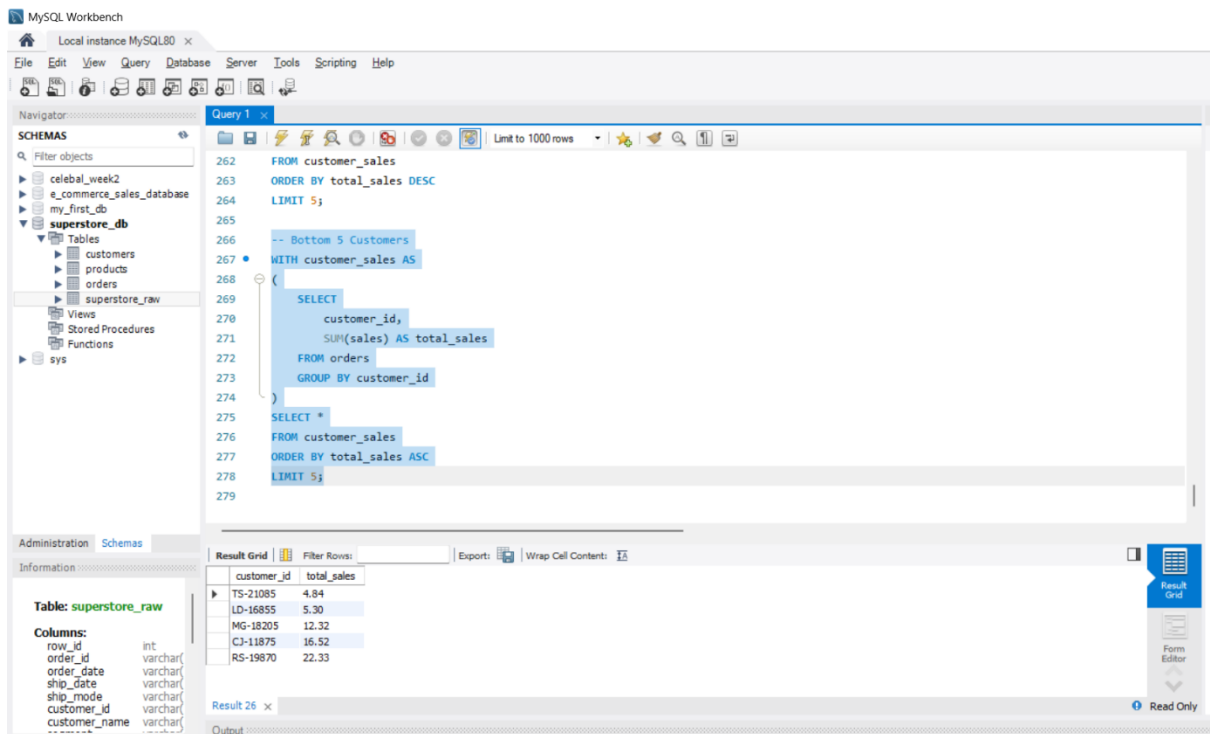
```
250
251 -- Mini Project
252 -- Top 5 Customers
253 WITH customer_sales AS
254 (
255     SELECT
256         customer_id,
257         SUM(sales) AS total_sales
258     FROM orders
259     GROUP BY customer_id
260 )
261 SELECT *
262 FROM customer_sales
263 ORDER BY total_sales DESC
264 LIMIT 5;
265
266
267
```

Below the query editor, the 'Result Grid' pane displays the results of the query. It shows a table with two columns: 'customer_id' and 'total_sales'. The results are sorted in descending order of total sales.

customer_id	total_sales
SM-20320	25043.07
TC-20980	19017.85
RB-19360	15117.35
TA-21385	14995.62
AB-10105	14355.61

- Customer sales are not evenly distributed across the dataset.
- A small group of customers contributes a significant portion of the overall revenue.
- Several customers placed only one order, indicating lower engagement levels.
- Customers with above-average sales play an important role in overall business performance.
- Window Functions proved useful for ranking customers and analyzing order-level data.

Bottom 5 Customers



The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'superstore_db' database with tables like 'customers', 'products', 'orders', and 'superstore_raw'. The main query editor contains the following SQL code:

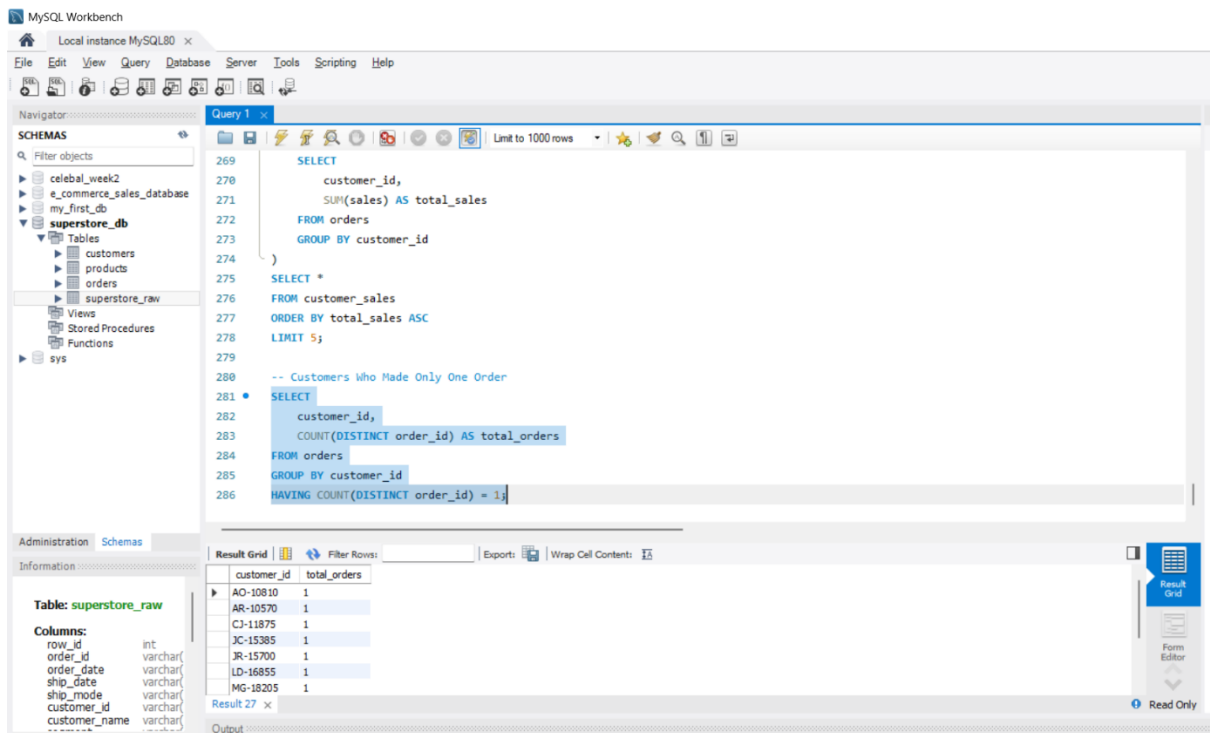
```
262 FROM customer_sales
263 ORDER BY total_sales DESC
264 LIMIT 5;
265
266 -- Bottom 5 Customers
267 WITH customer_sales AS
268 (
269     SELECT
270         customer_id,
271         SUM(sales) AS total_sales
272     FROM orders
273     GROUP BY customer_id
274 )
275 SELECT *
276 FROM customer_sales
277 ORDER BY total_sales ASC
278 LIMIT 5;
279
```

The bottom of the interface shows the 'Result Grid' with the following data:

customer_id	total_sales
TS-21085	4.84
LD-16855	5.30
MG-18205	12.32
CJ-11875	16.52
RS-19870	22.33

These customers generated the lowest sales and may represent occasional or low-engagement buyers.

Customers with Only One Order



The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'superstore_db' database with tables like 'customers', 'products', 'orders', and 'superstore_raw'. The main query editor contains the following SQL code:

```
269 SELECT
270     customer_id,
271     SUM(sales) AS total_sales
272 FROM orders
273 GROUP BY customer_id
274 )
275 SELECT *
276 FROM customer_sales
277 ORDER BY total_sales ASC
278 LIMIT 5;
279
280 -- Customers who Made Only One Order
281 SELECT
282     customer_id,
283     COUNT(DISTINCT order_id) AS total_orders
284 FROM orders
285 GROUP BY customer_id
286 HAVING COUNT(DISTINCT order_id) = 1;
```

The bottom of the interface shows the 'Result Grid' with the following data:

customer_id	total_orders
AO-10810	1
AR-10570	1
CJ-11875	1
JC-15385	1
JR-15700	1
LD-16855	1
MG-18205	1

Several customers placed only one order, indicating limited interaction with the business.

Customers with Above Average Sales

The screenshot shows the MySQL Workbench interface. The 'Query 1' window contains the following SQL code:

```
286 HAVING COUNT(DISTINCT order_id) = 1;
287
288 -- Customers With Above Average Sales
289 WITH customer_sales AS
290 (
291     SELECT
292         customer_id,
293         SUM(sales) AS total_sales
294     FROM orders
295     GROUP BY customer_id
296 )
297 SELECT *
298 FROM customer_sales
299 WHERE total_sales >
300 (
301     SELECT AVG(total_sales)
302     FROM customer_sales
303 );
```

The 'Result Grid' shows the following data:

customer_id	total_sales
BH-11710	6255.34
JM-15070	4930.49
PK-19075	8158.65
TB-21520	4730.62
MA-17560	4280.62
SN-20710	3127.97
LC-16930	4481.16

These customers performed better than the average customer and contribute strongly to overall revenue.

Highest Order Value per Customer

The screenshot shows the MySQL Workbench interface. The 'Query 1' window contains the following SQL code:

```
294 FROM orders
295 GROUP BY customer_id
296 )
297 SELECT *
298 FROM customer_sales
299 WHERE total_sales >
300 (
301     SELECT AVG(total_sales)
302     FROM customer_sales
303 );
304
305 -- Highest Order Value Per Customer
306
307 SELECT
308     customer_id,
309     MAX(sales) AS highest_order_value
310 FROM orders
311 GROUP BY customer_id;
```

The 'Result Grid' shows the following data:

customer_id	highest_order_value
CG-12520	731.94
DV-13045	721.88
SO-20335	957.58
BH-11710	1706.18
AA-10480	479.97
JM-15070	1819.86
HP-14615	1242.90

This analysis highlights the largest transaction made by each customer.

7. KEY LEARNINGS

- Learned how Subqueries can be used to compare records against calculated values such as averages and maximum sales.
- Understood how CTEs help break complex calculations into simpler and more readable steps.
- Gained hands-on experience with Window Functions like RANK() and ROW_NUMBER() for ranking and customer-level analysis.
- Learned how data normalization helps organize large datasets into meaningful tables such as customers, products, and orders.
- Improved understanding of how SQL can be used to answer real business questions rather than just retrieving data.

8. CHALLENGES FACED AND HOW THEY WERE RESOLVED

1. Product Data Duplication

While creating the products table, some Product IDs were found to be associated with multiple product descriptions. This caused issues while enforcing uniqueness in the products table.

Resolution:

The records were grouped using Product ID before insertion, ensuring that each Product ID appeared only once in the final table.

2. Query Performance Issues

While working on the highest sales order query, some query versions took longer to execute and occasionally caused connection timeout errors.

Resolution:

The query structure was simplified and optimized to obtain the required result without affecting correctness.

3. Data Organization

The raw dataset contained customer, product, and order information in a single table, making analysis difficult.

Resolution:

The dataset was normalized into separate customers, products, and orders tables, making the analysis more structured and easier to perform.

CONCLUSION

In this assignment, the Superstore dataset was analyzed using advanced SQL concepts including Subqueries, Common Table Expressions (CTEs), Window Functions, and Joins.

The dataset was first organized into separate tables and then used to answer various business questions related to customer sales performance. Through this analysis, it was possible to identify top customers, high-value transactions, customer rankings, and overall sales trends.

Overall, the assignment provided practical experience in applying SQL techniques to solve real-world business problems and generate meaningful insights from data.